Decomposicao geometrica de um semiprimo - Versao consolidada v14

August 21, 2026

Contents

1 Decomposição geométrica de um semiprimo N = L × q e sua relação com compressão de dados **3**

1.1 1. Motivação . . . 3
1.2 2. Construção geométrica . . . 3
1.3 3. Decomposição em T1 / T2 . . . 3
1.4 4. Leitura em termos de compressão . . . 4
1.5 5. Implementação . . . 4
1.6 6. Resultados . . . 4
1.7 7. Duas técnicas de truncamento sem divisão . . . 5
1.8 8. Fusão de blocos como grafo (union-find) . . . 5
1.9 9. A constante da Tabela 1 não é universal: sensibilidade ao alinhamento binário . . . 5
1.9.1 9.1 Correção: a causa não é a heurística de amostragem . . . 6
1.10 10. Hexágono: dois trapézios colados pela hipotenusa . . . 6
1.11 11. Efeito telescópico entre valores de k (gnomons generalizados) . . . 7
1.12 12. Comparação com dados de ocupação esparsa (esfera e terreno sintéticos) . . . 7
1.12.1 12.1 Por que dados sintéticos . . . 7
1.12.2 12.2 Extensão do método para 3D . . . 8
1.12.3 12.3 Resultados . . . 8
1.12.4 12.4 Interpretação . . . 8
1.12.5 12.5 O piso teórico para dados incompressíveis: um terceiro tipo de truncagem . . . 8
1.13 13. Custo amortizado: construção única vs. renderização por quadro . . . 9
1.13.1 13.1 Por que contagem de nós não é a métrica certa . . . 9
1.13.2 13.2 Metodologia . . . 9
1.13.3 13.3 Ponto de equilíbrio (break-even) . . . 10
1.13.4 13.4 Interpretação . . . 10
1.14 14. O triângulo como figura de referência (Altura Máxima e Projeção Quadrada) . . . 10
1.14.1 14.1 A projeção como quadrado . . . 10
1.14.2 14.2 O quadrado como referência de medida . . . 10
1.14.3 14.3 Consequência prática . . . 11

1.14.414.4 Nota conceitual: medir com referência não muda a figura . . 11
1.1515. Relações Métricas: Leitura Associativa vs. Fórmula 11
1.15.115.1 O teorema 11
1.15.215.2 O que essas relações realmente são: associação antes de fórmula 11
1.15.315.3 Por que essa distinção importa 11
1.1616. Recuperação Geométrico-Métrica: Método da Partição e Escala . . . 12
1.16.116.1 Premissas 12
1.16.216.2 Princípio da escala base 12
1.16.316.3 Recuperação por partição e truncamento 12
1.16.416.4 Síntese do algoritmo 12
1.16.516.5 Vantagem e generalização 12
1.1717. Próximos passos 13
1.1818. Status da implementação em engine (C++) 13
1.1919. Números ímpares, trapézios e tesselação (do “Registro de Progresso”) 15
1.19.119.1 Decomposição de ímpares e números triangulares (provado) 15
1.19.219.2 Gnômon: ímpares como crescimento de quadrados (confirmado) 16
1.19.319.3 Tesselação por losangos (confirmado — e verificado nesta consolidação) 16
1.19.419.4 Contar vs. medir (nota de correção do próprio registro, mantida) 17
1.19.519.5 Escala repetida: não é ponto de fuga projetivo, é relação log-linear (fechado, correção interna já feita no registro original) . . . 17
1.19.619.6 O que fica de fora desta seção 17
1.2020. Duas pontes com o documento “Repunits, Séries Geradoras, Tensores e Tesselação” 18
1.20.120.1 Recorrência de Newton para repunits (estende a Seção 7) . . 18
1.20.220.2 Tesselação por resíduo mod m: a teoria geral por trás da Seção 9.1 19
1.20.320.3 Correção real: medida uniforme vs. tesselação em múltiplas escalas (`BitmapQuadtree`) 19
1.20.420.4 Segunda correção: a pirâmide ainda era força bruta disfarçada 20
1.20.520.5 Resultado negativo: Morton code + busca binária não compensa aqui 20
1.20.620.6 A hipótese do m-tree (Seção 20.2), testada: confirmação parcial e delimitada 21
1.20.720.7 Ponto de fuga como caso particular de tesselação: por que a engine mede em vez de assumir 22
1.20.820.8 O scroll é um cilindro: homografia local, projeção cilíndrica global 22
1.2121. Números primos como interseção de triângulos: completando a Seção 19 23
1.21.121.1 Critério de unicidade e primalidade (completa a Seção 19.1) 23
1.21.221.2 Pirâmide simétrica e números hexagonais (complementar, não ligada a primalidade) 24
1.21.321.3 Vãos e partições: o mesmo conjunto de pontos, múltiplas tesselações 24
1.21.421.4 Do hexágono aos sólidos: cubo e octaedro (nova conexão, verificada) 24

1.21.521.5 Pontos de fuga (Seção 7 do documento): verificado de forma independente 25

1 Decomposição geométrica de um semiprimo N = L × q e sua relação com compressão de dados

Versão consolidada — funde o relatório principal (seções 1–14) com o adendo sobre o triângulo como figura de referência (seções 15–17), cuja numeração original colidia com a expansão do relatório principal.

1.1 1. Motivação

Um semiprimo $N = L \times q$ pode ser representado por uma malha triangular de pontos: para L fixo, os pontos (x, y) com $0 \le y < L$ e $0 \le x < L - y$ formam um triângulo retângulo de $L(L+1)/2$ pontos. Dentro dessa malha, um bloco quadrado de lado

$$k = L - q$$

concentra a informação sobre o fator q: conhecer k (e L) é equivalente a conhecer q. O objetivo deste relatório é registrar a construção geométrica feita, a decomposição observada em duas regiões em forma de T, e a analogia com técnicas de compressão/renderização que motivou a discussão seguinte.

1.2 2. Construção geométrica

A malha é desenhada em duas projeções: uma vista 2D direta e uma vista 3D em que o bloco central ganha altura k, sugerindo um paralelepípedo "extraído" da malha plana. Exemplo: malha triangular $L = 9$ com bloco central $k = 3$ ($q = L - k = 6$, $N = Lq = 54$).

1.3 3. Decomposição em T1 / T2

Centralizando o bloco de lado k sobre a diagonal de simetria $x = y$ da malha (posição $c = (L - k)/2$), o restante da malha se separa, ponto a ponto, em três classes:

- $T_1 = \{(x, y) : x > y\}$ — bloco próximo ao vértice $(L - 1, 0)$;
- $T_2 = \{(x, y) : y > x\}$ — bloco próximo ao vértice $(0, L - 1)$, espelho de T_1 pela troca $x \leftrightarrow y$;
- pontos com $x = y$ — a própria dobra de simetria.

Cada um desses blocos tem, visualmente, o formato de um T. Para $L = 9$, $k = 3$: $|T_1| = |T_2| = 18$ pontos e 3 pontos na diagonal, totalizando $45 = \binom{9}{2}$ pontos fora do bloco de 9 pontos (k^2).

1.4 4. Leitura em termos de compressão

A decomposição sugere descrever a malha sem enumerar todos os $L(L+1)/2$ pontos:

1. o bloco central é homogêneo, determinado por três números (L, k, c);
2. T_1 e T_2 são simétricos — basta descrever um e refletir;
3. a malha completa é reconstruída aplicando essas regras, sem processar a região redundante.

Essa é a mesma lógica de voxel culling, sparse octrees e clipmaps: evitar processar regiões homogêneas ou fora de interesse. A diferença é a origem do critério de redutibilidade — aqui algébrico (determinado por L e q); num motor gráfico, geralmente por esparsidade de conteúdo ou visibilidade.

1.5 5. Implementação

Implementado em Python (`compress_demo.py`):

- **Critério de homogeneidade:** região $[x_0, x_0 + s) \times [y_0, y_0 + s)$ é homogênea se os quatro cantos e o centro têm o mesmo valor; para $s \leq 2$ a checagem é exata célula a célula (necessário porque a hipotenusa é uma fronteira fina que pode escapar da amostragem por cantos).
- **Estrutura de dados:** quadtree — cada nó é folha (valor único) ou se subdivide em 4 filhos. Nós homogêneos nunca são expandidos.
- **Regra de reconstrução:** `query(x,y)` desce a árvore até a folha correspondente — nenhum array $L \times L$ é materializado em memória.

Para o caso 3D, o volume é processado fatia por fatia ao longo de z: cada fatia é comprimida em um quadtree independente e descartada antes da próxima (streaming) — pico de memória = tamanho de uma única fatia comprimida.

1.6 6. Resultados

L	k	células (L^2)	nós do quadtree	nós/L
16	5	256	89	5.56
64	21	4 096	401	6.27
256	85	65 536	1 673	6.54
1024	341	1 048 576	6 785	6.63
4096	1365	16 777 216	27 257	6.65

Tabela 1: Crescimento do quadtree com L (fixando k ≈ L/3). A razão nós/L estabiliza, confirmando escala O(L).

O experimento de streaming 3D (L=256, k=70) confirma o ganho de memória: volume bruto de 16,8 milhões de células, mas pico de apenas 1.561 nós de uma única fatia comprimida — razão $9{,}3 \times 10^{-5}$. Reconstrução ponto a ponto validada contra a função de ocupação original em 2000 amostras aleatórias por caso, sem divergência. **(Confirmado por execução direta do script `compress_demo-3.py`: pico de 1.561 nós bate exato.)**

1.7 7. Duas técnicas de truncamento sem divisão

Truncamento por máscara (resíduo): $i \,\&\, R_k$ (com $R_k = 2^k - 1$) calcula $i \bmod 2^k$ sem custo de divisão, mas não preserva a ordem de grandeza de i. Usado no protótipo original para evitar divisões/módulos.

Truncamento por deslocamento (shift): $i \gg k$ calcula $\lfloor i/2^k \rfloor$ exatamente, preservando a magnitude. Técnica correta quando o objetivo é dimensionar (decidir se dois valores são "parecidos o bastante").

Teste numérico ($k = 4$, $R_k = 15$): os testes $(i \,\&\, R_k) \leq 1$ e $(i \gg k) = 0$ discordam em 128 dos 256 casos — respondem perguntas diferentes; nenhum é "o errado".

1.8 8. Fusão de blocos como grafo (union-find)

Em vez de subdividir recursivamente, trata-se os blocos como nós de um grafo, fundindo blocos adjacentes cujo valor truncado (por deslocamento) coincide, via union-find — abordagem de Felzenszwalb–Huttenlocher, mais flexível que o quadtree (regiões fundidas não precisam ser quadrados alinhados à grade).

L	blocos totais	componentes	razão
32	64	2	0.0312
128	1 024	2	0.0020
512	16 384	2	0.0001
1024	65 536	2	0.00003

Tabela 2: Fusão de blocos por grafo (kshift=1, blocos 4×4). Componentes O(1), à custa de perder a fronteira exata. **(Confirmado por execução de `merge_grafo.py`.)**

Quadtree preserva a fronteira exata ao custo de O(L); o grafo com truncamento comprime para O(1) classes ao custo de perder a posição exata da fronteira.

1.9 9. A constante da Tabela 1 não é universal: sensibilidade ao alinhamento binário

A hipótese de que nós/L fosse função fechada apenas de k/L não bateu com os dados (discrepância de até ~46%). Variando k/L para $L = 4096$ fixo:

k/L	nós/L
1/4	4.00
1/6	5.32
1/3	6.66
3/8	4.01
1/2	4.00
5/8	4.01

Tabela 3: Não há tendência suave — proporções bem alinhadas em binário (1/4, 1/2) retornam à constante baixa ≈4; 1/3 é o pior caso observado.

Causa (hipótese original, ver correção abaixo na Seção 9.1): o teste de homogeneidade por amostragem (cantos + centro) funciona perfeitamente quando as

bordas do bloco caem sobre fronteiras de subdivisão (potências de 2), mas força subdivisões extras quando k e $c = (L - k)/2$ não são bem alinhados. $k/L = 1/3$ é mau alinhamento; $k/L \in \{1/4, 1/2\}$ é bem alinhado.

Consequência: não existe fórmula fechada simples nós$(L, k) = f(k/L) \cdot L$ válida para todo k. Uma afirmação anterior (nós$(L, k) = 3L - k$, de um experimento diferente com teste exato) foi removida por não corresponder a esta implementação. O crescimento O(L) permanece válido; a constante de proporcionalidade depende do alinhamento binário de k (ver Seção 9.1 — não da heurística de amostragem, como se pensava originalmente).

1.9.1 9.1 Correção: a causa não é a heurística de amostragem

Esta subseção corrige a Seção 9 original. A hipótese de que o viés vinha de falsos positivos da amostragem por cantos+centro foi testada diretamente: implementou-se um teste de homogeneidade **geométrico exato** (interseção analítica reta-retângulo entre a região e as fronteiras conhecidas — hipotenusa e bordas do bloco; ver o script de validação e o módulo correspondente na engine), que por construção nunca produz falso positivo.

Resultado: o teste exato produz **exatamente os mesmos números de nós** que a amostragem heurística, para todos os casos testados — a Tabela 1 inteira (L=16 a 4096) e a Tabela 3 inteira (k/L de 1/6 a 5/8), com diferença zero em todos os casos de contagem de nós, e diferença $\leq 0{,}005$ em nós/L nos casos fracionários (arredondamento).

k/L	heurística (nós/L)	exato (nós/L)	diferença
1/4	4,000	4,001	0,001
1/6	5,320	5,323	0,003
1/3	6,660	6,655	-0,005
3/8	4,010	4,006	-0,004
1/2	4,000	4,000	0,000
5/8	4,010	4,014	0,004

Conclusão corrigida: a amostragem por cantos+centro já era, na prática, exata para esta geometria específica (fronteiras retas simples: uma diagonal e um quadrado axis-aligned). A sensibilidade de nós/L ao alinhamento de k/L **não é um artefato da heurística de amostragem** — é uma propriedade geométrica real de como a posição do bloco central interage com a grade de subdivisão binária do quadtree (potências de 2). Frações bem alinhadas a potências de 2 ($k/L \in \{1/4, 1/2\}$) sempre vão gerar menos nós que frações mal alinhadas ($k/L = 1/3$), **mesmo com um teste de homogeneidade perfeito**. Validado por reconstrução ponto a ponto (0 falhas em 3000 amostras aleatórias, múltiplos (L, k)) e por paridade C++/Python.

1.10 10. Hexágono: dois trapézios colados pela hipotenusa

Removendo o triângulo menor de lado k (a "ponta") do triângulo de lado L, sobra um trapézio T_1 (base curta $x + y = k$, base larga $x + y = L - 1$). Refletindo T_1 pela

hipotenusa — $(x, y) \mapsto (C - y, C - x)$, $C = L - 1$ — obtém-se T_2, colado a T_1 pela base larga compartilhada: um hexágono simétrico.

$$|T_1| = \sum_{i=k+1}^{L} i = \frac{L(L+1) - k(k+1)}{2}$$

$$N_{hex}(L, k) = 2|T_1| - L = L^2 - k(k+1)$$

Validado contra construção explícita para 30 combinações (L, k), $L \in [5, 100]$ — todos exatos:

L	k	pontos (construção real)	$L^2 - k(k+1)$
9	3	69	69
20	9	310	310
50	24	1 900	1 900
100	49	7 550	7 550

Leitura geométrica direta: L^2 é a malha quadrada completa; $k(k+1) = k^2 + k$ são as duas cunhas triangulares retiradas dos cantos — o hexágono é "o quadrado $L \times L$ menos duas pontas".

1.11 11. Efeito telescópico entre valores de k (gnomons generalizados)

Fixando L e variando k:

$$N_{hex}(L, k) - N_{hex}(L, k+1) = 2(k+1)$$

Análogo direto dos números gnomon clássicos. A soma das cascas de $k = 0$ até $k = m - 1$ reconstrói o salto direto:

$$\sum_{k=0}^{m-1} 2(k+1) = m(m+1) = N_{hex}(L, 0) - N_{hex}(L, m)$$

Confirmado numericamente (L=100, m=10: soma das cascas = 110, salto direto = 110).

1.12 12. Comparação com dados de ocupação esparsa (esfera e terreno sintéticos)

1.12.1 12.1 Por que dados sintéticos

Sem acesso a datasets reais neste ambiente, usam-se dados sintéticos que reproduzem características estruturais: esfera num cubo L^3 (proxy de órgão/tumor, fronteira curva suave), terreno por heightmap suave (proxy de voxels de jogo), e ruído puro como controle.

1.12.2 12.2 Extensão do método para 3D

O quadtree é generalizado para octree: nó cúbico de lado s subdivide em 8 filhos de lado $s/2$ quando não homogêneo (mesmo critério de amostragem, exato para $s \leq 2$).

1.12.3 12.3 Resultados

L	nós (esfera)	nós/L²	nós (terreno)	nós/L²
16	905	3.54	1 225	4.79
32	3 641	3.56	4 713	4.60
64	14 377	3.51	22 489	5.49
128	57 769	3.53	93 585	5.71

Tabela 5: A razão nós/L² estabiliza para ambos, confirmando O(L²). **(Confirmado por execução direta de `octree3d.py` — bate exato em todas as linhas.)**

Construção	Dimensão	Crescimento	Razão nós/bruto
Malha algébrica (Seção 5)	2D	O(L)	0,052
Esfera sintética	3D	O(L²)	0,028
Terreno sintético	3D	O(L²)	0,045
Ruído puro (sem estrutura)	3D	O(L³)	1,123 (pior que bruto!)

Tabela 6: Comparação final. ***(Confirmado: 0.0275≈0.028; 0.0446≈0.045; 1.1234≈1.123; malha algébrica 0.0501≈0.052 para L=128.)***

1.12.4 12.4 Interpretação

O ganho em dados sintéticos "realistas" é da mesma ordem de grandeza do obtido na construção algébrica. O princípio (compressão por homogeneidade local, custo proporcional à fronteira) generaliza para qualquer dado com fronteiras bem comportadas — não é peculiaridade do semiprimo.

O ruído puro é o contraponto: sem estrutura a explorar, o octree piora em relação ao array bruto (razão > 1), porque cada subdivisão adiciona um nó interno sem atingir folha homogênea maior que $1 \times 1 \times 1$.

1.12.5 12.5 O piso teórico para dados incompressíveis: um terceiro tipo de truncagem

A razão > 1 para ruído puro não é artefato — é limite matemático exato. Para dados verdadeiramente incompressíveis, a árvore desce até folhas $1 \times 1 \times 1$ em todo lugar:

$$N(d) = \sum_{i=0}^{d} b^i = \frac{b^{d+1} - 1}{b - 1}, \quad b^d = \text{raw}$$

$$\frac{N(d)}{b^d} = \frac{b}{b-1} - \underbrace{\frac{b^{-d}}{b-1}}_{\text{termo de truncagem}} \xrightarrow{d\to\infty} \frac{b}{b-1}$$

Para octree ($b = 8$): limite $8/7 \approx 1{,}142857$. Medido em $L = 8$ ($d = 3$) com verificação exata de homogeneidade: $1{,}142578$ — bate a fórmula exatamente.

Nota de validação (não presente no relatório original, verificada nesta consolidação): essa igualdade exata só se sustenta com dados verdadeiramente incompressíveis — em teste com valores contínuos (float aleatório, sem chance de coincidência), a medição bate $1{,}142578$ exatamente, confirmado também para $d = 1..5$ contra a fórmula fechada e contra o termo residual $b^{-d}/(b-1)$, todos exatos. **Com dados binários (0/1, que é o que `make_noise_occ` de fato gera)**, a razão medida fica sistematicamente **abaixo** do piso teórico (testado: $1{,}126953$ em vez de $1{,}142578$ para os mesmos parâmetros), porque blocos de 8 células têm ~0,8% de chance de sair homogêneo por coincidência mesmo sendo ruído puro. Isso não invalida a fórmula — ela descreve o pior caso absoluto — mas o valor específico citado como "medido" no relatório pressupõe um alfabeto de valores efetivamente contínuo, não o gerador binário incluído em `octree3d.py`.

Essa truncagem é qualitativamente diferente das da Seção 7: não é escolha de projeto, é consequência inevitável de qualquer representação finita — um viés sistemático, quantificável e residual, presente em qualquer estrutura em árvore aplicada a dados finitos.

1.13 13. Custo amortizado: construção única vs. renderização por quadro

1.13.1 13.1 Por que contagem de nós não é a métrica certa

O caso de uso real (renderização, fatiamento de volumes) não é sobre armazenar — é sobre visitar/pintar o volume repetidamente, quadro a quadro.

1.13.2 13.2 Metodologia

Compara-se **array denso** (percorre todas as L^3 células do zero a cada quadro) contra **octree** (construído uma vez; cada quadro seguinte apenas percorre a árvore pronta).

Cenário	build (1×)	denso/quadro	árvore/quadro	ganho/quadro
Esfera L=32	20,7 ms	9,3 ms	0,32 ms	29×
Terreno L=32	76,1 ms	46,0 ms	0,43 ms	107×
Esfera L=64	78,0 ms	77,8 ms	1,54 ms	51×
Terreno L=64	383,9 ms	357,9 ms	2,39 ms	150×

Tabela 7: Python puro, 20 quadros por cenário. **(Confirmado por execução de `render_bench2.py` — mesma ordem de grandeza e conclusão qualitativa; tempos absolutos variam por máquina, como esperado.)**

1.13.3 13.3 Ponto de equilíbrio (break-even)

No cenário mais custoso (terreno, L=64): build (384 ms) equivale a pouco mais de 1 quadro de array denso (358 ms/quadro) — a partir do segundo quadro a árvore já está adiante. Em todos os cenários, custo total (build + 20 quadros) ficou entre 4 e 16× menor que o array denso.

1.13.4 13.4 Interpretação

O ganho não vem de acesso mais rápido a um ponto isolado (é O(log L) na árvore contra O(1) no array denso — mais lento nessa operação específica). Vem do reuso amortizado de um trabalho de homogeneização pago uma única vez, ao longo de muitos quadros sobre um dado que não muda a cada instante. É por isso que motores gráficos constroem a estrutura esparsa ao carregar a cena e reaproveitam em todos os quadros seguintes.

1.14 14. O triângulo como figura de referência (Altura Máxima e Projeção Quadrada)

(Seções 14–16 vêm do adendo original — lá numeradas 13–15 — renumeradas aqui para encaixar após a expansão do relatório principal.)

Entre as figuras derivadas da malha (triângulo, trapézio, hexágono, ampulheta), o triângulo original — antes de qualquer corte ou dobra — é o de maior altura. Todas as demais figuras (Seção 10) são obtidas subtraindo ou recombinando partes do triângulo original; como nenhuma operação acrescenta pontos novos fora da malha de partida, nenhuma pode superar a altura do triângulo original — ele é o teto geométrico do qual as outras derivam.

1.14.1 14.1 A projeção como quadrado

O triângulo retângulo de catetos L tem como bounding box exatamente o quadrado $L \times L$: a projeção sobre cada eixo cobre $[0, L)$, então a menor região retangular capaz de conter o triângulo é o quadrado de área L^2. O próprio triângulo ocupa metade dessa projeção:

$$|T_{\triangle}| = \frac{L(L+1)}{2} \approx \frac{L^2}{2}$$

com a diferença de $L/2$ vindo dos pontos da própria hipotenusa discreta, contados uma única vez.

1.14.2 14.2 O quadrado como referência de medida

Essa projeção fornece uma unidade de medida comum: em vez de medir cada figura isoladamente, mede-se sua diferença em relação ao quadrado L^2 da figura de maior altura. Exatamente a leitura já obtida algebricamente na Seção 10 para o hexágono:

$$N_{hex}(L, k) = L^2 - k(k+1)$$

— o quadrado funcionando como referência absoluta, com as figuras derivadas descritas por subtração, não recontadas ponto a ponto.

1.14.3 14.3 Consequência prática

Ao tomar a figura de maior altura como referência de projeção, medir um conjunto de figuras relacionadas se reduz a uma quantidade fixa (L^2) mais termos de correção específicos de cada corte ou dobra — mesmo princípio de eficiência da Seção 4.

1.14.4 14.4 Nota conceitual: medir com referência não muda a figura

Usar L^2 como referência não altera a figura medida — o hexágono continua tendo exatamente $L^2 - k(k+1)$ pontos, quer descrito por subtração ou contagem direta. O que muda é apenas o custo de obter a contagem: com a referência, uma subtração; sem ela, enumeração região por região. A figura é a mesma nos dois casos; o ganho é inteiramente no processo de cálculo, não no objeto calculado.

1.15 15. Relações Métricas: Leitura Associativa vs. Fórmula

1.15.1 15.1 O teorema

Num triângulo retângulo, a altura relativa à hipotenusa o divide em dois triângulos menores, semelhantes entre si e ao original. Sejam b, c os catetos, a a hipotenusa, h a altura relativa a a, m, n as projeções de c e b sobre a ($m + n = a$):

$$b^2 = a \cdot n, \quad c^2 = a \cdot m, \quad h^2 = m \cdot n, \quad a \cdot h = b \cdot c$$

Somando as duas primeiras: $b^2 + c^2 = a(m+n) = a^2$ — o Teorema de Pitágoras emerge da semelhança, não como ponto de partida independente.

1.15.2 15.2 O que essas relações realmente são: associação antes de fórmula

$b^2 = a \cdot n$ não é, na origem, uma sentença algébrica a memorizar — é a tradução de uma percepção visual: o quadrado de lado b tem a mesma área que o retângulo de lados a e n, percebida pelo encaixe dos triângulos semelhantes. A fórmula é o registro posterior de uma associação já vista.

Mesmo tipo de leitura que perpassa todo o relatório: perceber que o hexágono é o quadrado menos duas pontas (Seção 10); perceber que o triângulo, por ter maior altura, projeta naturalmente o quadrado L^2 (Seção 14) — a mesma leitura associativa aplicada à figura de referência do relatório inteiro.

1.15.3 15.3 Por que essa distinção importa

Reconhecer que a fórmula é tradução, não fonte, muda a ordem natural do trabalho: buscar primeiro a associação visual (semelhança, encaixe, projeção), e só então traduzir em símbolos para comunicar ou verificar.

1.16 16. Recuperação Geométrico-Métrica: Método da Partição e Escala

1.16.1 16.1 Premissas

Dadas a área A de um retângulo (ou $A_T = A/2$ do triângulo correspondente) e sua diagonal D, existe único par $a \geq b > 0$ com $a \cdot b = A$, $a^2 + b^2 = D^2$. Objetivo: recuperar a e b sem a equação quadrática explícita $x^2 - Sx + A = 0$.

1.16.2 16.2 Princípio da escala base

Usa-se um triângulo de referência (u, v, w) com propriedades conhecidas. Se a figura real se relaciona com a referência por fator de escala k, a área escala quadraticamente: Área real $= k^2 \cdot A_0$. Escolhendo $w = D$, tem-se $k = 1$, eliminando a necessidade de extrair raiz para obter o fator de escala.

1.16.3 16.3 Recuperação por partição e truncamento

Altura relativa à diagonal: $h_D = \dfrac{2A_T}{D} = \dfrac{A}{D}$ — medida linear por divisão simples, sem raízes.

Invariante do quadrado da soma: $(a + b)^2 = D^2 + 2A$ — recupera $(a + b)$ sem calcular a, b separadamente. Perímetro: $P = 2(a + b) = 2\sqrt{D^2 + 2A}$.

1.16.4 16.4 Síntese do algoritmo

Passo	Operação geométrica	Expressão	Exemplo (A=60, D=13)
1	Altura relativa à diagonal	$h_D = A/D$	$60/13$
2	Quadrado composto	$Q = D^2 + 2A$	$169 + 120 = 289$
3	Semiperímetro	$S = \sqrt{Q}$	17
4	Perímetro total	$P = 2S$	34

Validado em C++ e Python nesta consolidação: bate exato o exemplo do documento ($a = 12$, $b = 5$, $P = 34$) e passa em 2000/2000 casos aleatórios (a única "falha" observada em teste anterior foi ruído de ponto flutuante de $\sim 10^{-6}$ no caso degenerado $a = b$, não erro do método). Já integrado ao módulo `Render::BoundingReference` da engine.

1.16.5 16.5 Vantagem e generalização

O método evita montar a equação de segundo grau, substituindo-a por análise de proporção, restrições lineares e decomposição por partições — mesma linha da Seção 15: a raiz quadrada, quando aparece (Passo 3), surge de reconhecer um quadrado perfeito por partição ($289 = 17^2$), não de extrair raiz arbitrária. Generaliza para qualquer prisma ou volume cuja base seja parametrizada por projeções e razões fixas de área — a mesma lógica de escala (k^2 para área, k^3 para volume) que rege semelhança de figuras em qualquer dimensão.

1.17 17. Próximos passos

A implementação atual usa teste de homogeneidade por amostragem (cantos + centro), suficiente para fronteiras retas simples mas não generalizável a fronteiras curvas ou regiões sobrepostas. Direções de continuação, com status atualizado nesta consolidação:

- **Testes de homogeneidade exatos** via geometria analítica (interseção reta-retângulo) — [OK] **implementado e validado** (`py/homogeneidade_exata.py`, `render::SpatialPart` na engine). Resultado inesperado: não elimina o viés de alinhamento binário descrito na Seção 9 — confirma que esse viés é geométrico real, não artefato da amostragem (ver Seção 9.1, correção).
- **Extensão a octree 3D completo** — [OK] **concluído** (Seção 12, `octree3d.py`), embora ainda usando o mesmo teste de homogeneidade por amostragem (não geométrico exato) da Seção 5.
- **Esquema híbrido** (grafo de fusão da Seção 8 para classificação grosseira O(1), refinado por quadtree da Seção 5) — [OK] **implementado** (`src/stream/hybrid_partition.hpp` da engine, ver Seção 18) e **testado com resultado negativo**: para a malha do relatório (L=256, k=85), o híbrido usa **2010 nós contra 1673 do quadtree puro** — pior, não melhor. Causa: a fusão grosseira separa "bloco central" (bounding box pequena) de "resto da malha" (T1+T2 nos dois cantos opostos), mas a bounding box desse segundo componente degenera para quase o tamanho da malha inteira, anulando o ganho do Passo 1. O híbrido só compensa quando os componentes fundidos são geometricamente compactos — não é o caso desta construção específica.
- **Dados de ocupação esparsa realistas** (voxels de jogos, volumes médicos) — parcialmente endereçado: dados sintéticos (esfera, terreno) implementados e validados (Seção 12); dados reais permanecem como continuação, por falta de acesso à rede neste ambiente.
- **Validação do piso teórico para dados incompressíveis (Seção 12.5)** — a fórmula está confirmada matematicamente, mas o script de geração de ruído (`make_noise_occ`) usa alfabeto binário, que produz medições sistematicamente abaixo do piso teórico por coincidências de homogeneidade local; um gerador de valores contínuos seria necessário para reproduzir o valor específico citado como "medido" no relatório.

1.18 18. Status da implementação em engine (C++)

Paralelamente à validação em Python (Seções 5–13), a mesma matemática foi portada para uma engine em C++20 (`engine-fluxo/src/`), estruturada como um Render Loop central que consome os módulos abaixo em ordem fixa a cada quadro. O objetivo desta seção é manter rastreável a relação entre teoria (este relatório) e implementação — cada módulo abaixo cita a seção de origem, e o `README.md` do repositório mantém o log cronológico de mudanças.

Módulo C++	Seção de origem	Status
`core::FastMath` (`TruncaPorMascara`, `TruncaPorShift`)	Seção 7	[OK] implementado, testado
`stream::UnionFind`	Seção 8	[OK] implementado, réplica de `merge_grafo.py`
`stream::SpatialPartition` (Quad/Octree, heurística)	Seções 5, 12	[OK] implementado
`stream::SpatialPartitionExact` (geométrico exato)	Seções 9, 17	[OK] implementado, paridade total com a heurística (ver Seção 9.1)
`stream::BitmapQuadtree` + `BitmapPyramid` (dado real)	Seção 20 (bunny)	[OK] O(1) por nó via mipmap booleano, medido (não suposto); paridade C++/Python 6/6 fatias
`stream::HybridPartition`	Seção 17	[OK] implementado, resultado negativo confirmado (ver acima)
`render::MetricRecovery`	Seção 16	[OK] implementado, 0 falhas em 2000 casos aleatórios (C++)
`render::BoundingReference` (L^2 − correção)	Seção 13	[OK] implementado, usa `MetricRecovery`
`stream::WindowController` (ring buffer)	— (arquitetura)	[OK] implementado
`core::HardwareProfiler`	— (arquitetura)	[OK] mede RAM real via `/proc/meminfo` (proxy de VRAM, documentado); PCIe ainda hardcoded (sem GPU neste ambiente)
`render::ProjectivePipeline`	— (arquitetura)	[OK] framebuffer real por software (não GPU); loop roda 60 frames reais; câmera aceita input real de teclado (v19) com fallback simulado
`render::VanishingPointAnalyzer`	Seção 19.5, 20.6	[OK] mede padrão linear vs log-linear entre fatias, valida contra -5,7708 já publicado
`render::Homography`	Seção 20.8 (nova)	[OK] rotação pura, validada (composição, degeneração perto de 90° medida)

Módulo C++	Seção de origem	Status
`render::CylindricalPhotoCompiler`	Seção 20.8 (nova)	[OK] compila o anel via projeção cilíndrica; exclusão mútua com Draw() confirmada por contagem
`stream::SliceStreamer`	Seções 5, 9, 20	[OK] lê `WFLX` (sintético) ou `WFLB` (bitmap real, ex. bunny) via seek real, detecção automática de formato
Teste de homogeneidade geométrico exato	Seção 9	[OK] implementado — ver correção da Seção 9.1

Todos os módulos marcados [OK] compilam com `g++ -std=c++20` e têm resultado numérico conferido contra os scripts Python correspondentes. O relatório completo de mudanças, incluindo o que foi recuperado após perda parcial de contexto entre sessões, está em `engine-fluxo/README.md`.

1.19 19. Números ímpares, trapézios e tesselação (do "Registro de Progresso")

Esta seção incorpora o que foi estabelecido no documento de notas de trabalho paralelo ("Registro de Progresso: Trapézios de Ímpares e Escala Projetiva"), filtrando o que está provado/confirmado por construção do que ficou como especulação em aberto (a Rota B de "ponto de fuga projetivo", que o próprio registro já havia corrigido — ver 19.5).

1.19.1 19.1 Decomposição de ímpares e números triangulares (provado)

Todo ímpar $N = 2k + 1$ se decompõe de forma única em dois inteiros consecutivos, $N = k+(k+1)$, e essa decomposição é exatamente a faixa entre duas linhas consecutivas de um triângulo maior: com $T_n = n(n + 1)/2$,

$$N = T_{k+1} - T_{k-1}.$$

Nota (completada na Seção 21.1 desta consolidação): essa decomposição é única precisamente quando N é primo — um critério de primalidade que não havia sido estabelecido aqui originalmente. Ver Seção 21 para o teorema completo, com verificação exaustiva.

Trapézios adjacentes (ex. $N = 5$ e $N = 7$) compartilham uma linha do triângulo ao empilhar — para $N \in \{5, 7\}$ isso reconstrói $T_4 = 10$ empilhando as 4 primeiras linhas. Essa é a mesma lógica de números figurados já usada nas Seções 3 e 10 deste relatório (a malha do semiprimo também é lida como diferença entre figuras de referência), o que confirma que os dois registros de trabalho compartilham o mesmo princípio de fundo — medir por diferença contra uma figura de referência, em vez de contar ponto a ponto.

1.19.2 19.2 Gnômon: ímpares como crescimento de quadrados (confirmado)

$$(k+1)^2 - k^2 = 2k + 1 = N.$$

Geometricamente, N é a borda em L (gnômon) que soma a um quadrado de lado k para fazê-lo crescer até lado $k+1$. Combinado com a Seção 19.1: o mesmo N que é soma de dois consecutivos é também a casca que expande um quadrado em um passo — duas rotas geométricas independentes (camadas quadradas concêntricas vs. faixa de triângulo) convergindo no mesmo número, o que é evidência de robustez, não coincidência.

1.19.3 19.3 Tesselação por losangos (confirmado — e verificado nesta consolidação)

Dois triângulos equiláteros colados por uma aresta formam um losango de ângulos 60°/120°, que tesselam o plano sem lacunas nem sobreposições. Sobre a mesma malha (sem nenhuma aresta nova), uma regra de cor por distância ao centro faz emergir uma forma maior que não estava desenhada explicitamente — emergência de forma a partir de uma regra aplicada a peças já existentes.

Verificação adicional feita nesta consolidação (não estava no registro original): confirmamos por dois métodos independentes que a construção é válida, não apenas plausível visualmente.

Método 1 — fechamento angular exato. Por divisibilidade de 360° pelo ângulo interno, apenas 3 polígonos regulares tesselam sozinhos: triângulo (60°, 6 por vértice), quadrado (90°, 4 por vértice) e hexágono (120°, 3 por vértice) — pentágono, heptágono e todos os demais testados até 12 lados falham. Para o losango do registro: vértice largo (120°) fecha com exatamente 3 losangos, vértice estreito (60°) fecha com exatamente 6 — ambos exatos, confirmando a Figura 3 do registro. Verificamos também as 11 configurações semi-regulares clássicas (misturas de polígonos, ex. $3.6.3.6$, a família do losango), todas fechando 360° exato.

Método 2 — rasterização independente (prova por amostragem densa, não por ângulo). Geramos a malha triangular via a transformação de cisalhamento padrão ($x = \ell(i + j/2)$, $y = hj$) e testamos uma amostra dessa de pontos contra uma lista de triângulos gerada **independentemente** (não pela mesma fórmula usada para classificar os pontos, o que seria circular). Resultado: 0% de lacuna, 99,3% dos pontos cobertos por exatamente 1 triângulo, 0,67% cobertos por 2 (esperado: pontos caindo exatamente sobre arestas compartilhadas, contados nos dois triângulos vizinhos). Losango, trapézio (3 triângulos) e hexágono (6 triângulos) tesselam porque são recortes exatos da mesma malha triangular já confirmada — herdam a cobertura por construção (partição de uma partição).

Nota de honestidade do processo: a primeira tentativa de rasterização teve um bug real (74,5% de cobertura, malha errada por zigue-zague codificado incorretamente linha a linha); a segunda tentativa de correção virou um teste vazio que sempre "passava" sem testar nada. Só a terceira versão (cisalhamento padrão + lista de triângulos gerada independentemente para o teste) produziu um resultado confiável. Isso é registrado aqui porque é relevante ao próprio princípio do documento: verificar por construção, não assumir que uma figura "parece" tesselar.

Limite explícito: essa verificação vale para peças que são recortes de uma malha que já tesselava (losango, trapézio, hexágono grande — todos sub-produtos da malha

triangular). Não generaliza para qualquer peça composta arbitrária: dois quadrados colados em formato de "L", por exemplo, não tesselam sozinhos da mesma forma simples.

1.19.4 19.4 Contar vs. medir (nota de correção do próprio registro, mantida)

Sobre a malha de losangos, medir área é soma finita do número de peças vezes a área de uma peça constante — sem cálculo, limites ou integrais. Essas ferramentas só seriam necessárias se a fronteira da forma fosse uma curva exata (não a aproximação em degraus que a malha produz) ou se a densidade variasse peça a peça de forma contínua. Essa distinção — o que mudou é o custo do cálculo, não o objeto medido — é a mesma já registrada na Seção 13.4 deste relatório (nota conceitual sobre medir com referência), reforçando que os dois documentos de trabalho convergem no mesmo princípio metodológico.

1.19.5 19.5 Escala repetida: não é ponto de fuga projetivo, é relação log-linear (fechado, correção interna já feita no registro original)

O registro testou se uma escala geométrica repetida ($x_i = x_0 r^i$, $y_i = y_0 - di$) gerava retas convergindo a um ponto de fuga projetivo clássico. O próprio teste (inclinação entre pontos consecutivos: $4{,}0 \to 2{,}0 \to 1{,}0$, valores diferentes a cada segmento) já refuta isso — os pontos não são colineares no plano (x, y), é uma curva exponencial. Aplicando $\ln(x)$ no eixo horizontal, os mesmos pontos ficam exatamente sobre uma reta, com inclinação constante ($-5{,}7708$), o que é esperado algebricamente: eliminando i de $y = y_0 - di$ e $\ln x = \ln x_0 + i \ln r$,

$$y = y_0 - \frac{d}{\ln r}(\ln x - \ln x_0),$$

reta em $\ln x$. **Mantido como estava no registro** — é um resultado mais preciso que a intuição visual original de "ponto de fuga", e o próprio documento já havia feito essa autocorreção antes de chegar aqui; não há nada a revisar nesta consolidação além de registrar que a autocorreção é matematicamente sólida (relação exponencial vira linear em escala log, o mesmo princípio de um gráfico log-linear ou espiral logarítmica).

1.19.6 19.6 O que fica de fora desta seção

O registro original marca os Passos 1–7 como fechados/confirmados/provados. Todos os seis pontos acima (19.1–19.5, cobrindo os Passos 1, 2, 4, 5, 6 e 7 do registro) foram incorporados. O Passo 3 (empilhamento visual de trapézios) é consequência direta de 19.1 e não introduz conteúdo matemático novo além do já registrado ali. Nenhuma parte do registro foi descartada por estar errada — a única correção em jogo (Rota B / ponto de fuga) já havia sido feita pelo próprio autor do registro antes desta consolidação.

1.20 20. Duas pontes com o documento “Repunits, Séries Geradoras, Tensores e Tesselação”

Esse documento (892 linhas) é majoritariamente **uma linha de pesquisa separada** — tesselação por resíduo mod m generalizada, Wang tiles, tetraedro de Pascal, teoria de Galois da quíntica, LFSR/de Bruijn —, sem relação direta com a construção do semiprimo $N = L \times q$ deste relatório. A maior parte não foi incorporada aqui por não pertencer a este projeto, não porque esteja errada. Duas partes, porém, conectam de forma específica e genuína com o que já temos:

1.20.1 20.1 Recorrência de Newton para repunits (estende a Seção 7)

A Seção 7 deste relatório já trata repunits ($R_k = 2^k - 1$) no contexto de truncamento por máscara. O documento traz um resultado adicional genuíno: repunits em qualquer base b satisfazem a recorrência linear

$$R_n = (b+1)R_{n-1} - bR_{n-2}, \quad R_0 = 0,\ R_1 = 1,$$

que nunca divide — ao contrário da fórmula fechada $(b^n - 1)/(b - 1)$, que falha em aritmética modular quando $b - 1$ não é invertível módulo m.

Verificado nesta consolidação: a recorrência bate exatamente contra o cálculo direto para $b \in \{2, 7, 10, 16\}$, $n = 2, \ldots, 7$; e reproduz corretamente R_n mod 6 para $b = 7$ (onde $b-1 = 6$ não tem inverso módulo 6, caso em que a fórmula fechada simplesmente não se aplica, mas a recorrência continua funcionando por não envolver divisão).

A parte que faltava: fast doubling combina velocidade com robustez. Como R_n é combinação linear de duas raízes características (1 e b), admite identidades de duplicação análogas às de Fibonacci:

$$R_{2n} = R_n(b^n + 1), \qquad R_{2n+1} = bR_{2n} + 1.$$

Isso reduz o custo de $O(n)$ passos (recorrência linear, um por n) para $O(\log n)$ multiplicações — sem nunca dividir, ao contrário da fórmula fechada.

Verificado nesta consolidação: 0 falhas contra o cálculo direto em 4 bases ($b = 2, 7, 10, 16$), $n = 0, \ldots, 49$; para $n = 1000$ ($b = 10$), contagem real de multiplicações deu **28**, contra $2\log_2(1000) \approx 19{,}9$ esperado teoricamente (mesma ordem de grandeza — a diferença vem de multiplicações auxiliares de contabilidade na implementação) e contra **1000 passos** da recorrência linear simples — redução real de ordem de grandeza, não apenas teórica.

Síntese dos três métodos:

Método	Custo	Funciona em aritmética modular geral?
Fórmula fechada $(b^n - 1)/(b - 1)$	$O(\log n)$ + 1 divisão	Não (falha se $b - 1$ não invertível)
Recorrência de Newton, passo a passo	$O(n)$, sem divisão	Sim
Fast doubling da recorrência de Newton	$O(\log n)$, sem divisão	Sim

A combinação (fast doubling + recorrência sem divisão) dá o melhor dos dois mundos: velocidade da fórmula fechada **e** robustez da recorrência em contexto modular — relevante para o mesmo tipo de cenário de `Core::FastMath` (Seção 18) onde hardware favorece multiplicação sobre divisão e nem sempre há inverso modular conveniente disponível.

Relevância prática: isso é diretamente aplicável ao `core::FastMath` da engine (Seção 18) se algum cálculo envolvendo repunits precisar rodar em aritmética modular (ex.: índices de buffer circular com m não potência de 2) — a recorrência (ou seu fast doubling, se n for grande) seria a rota correta, não a fórmula fechada.

1.20.2 20.2 Tesselação por resíduo mod m: a teoria geral por trás da Seção 9.1

A Seção 9.1 deste relatório (correção sobre o viés de alinhamento binário do quadtree) descobriu empiricamente que a razão nós/L depende de k/L estar alinhado a potências de 2 — propriedade geométrica real, não artefato de amostragem. O documento formaliza exatamente o mecanismo geral por trás disso: um segmento entre a e b admite m-seção em pontos de rede se e somente se $a \equiv b \pmod{m}$, e em d dimensões o espaço se tesselam em m^d classes de resíduo.

A conexão precisa: o quadtree subdivide por potências de 2 — ou seja, é uma tesselação com $m = 2$. A Seção 9.1 mostrou que $k/L = 1/4$ e $1/2$ (frações que são potências de 2 exatas) alinham bem com essa tesselação, enquanto $k/L = 1/3$ não alinha com nenhuma potência de 2 — esse é precisamente o fenômeno que a tesselação mod m do documento descreve de forma geral: alinhamento com a classe de resíduo do módulo usado pela estrutura de subdivisão (aqui, $m = 2$) determina se a fronteira cai exatamente sobre uma linha de corte ou força subdivisão extra.

O que isso sugere como próximo passo: se `SpatialPartition` usasse subdivisão em m partes (não necessariamente 2) escolhido para alinhar com o k/L típico dos dados, a constante de compressão poderia melhorar — mas isso mudaria a estrutura de dados (quadtree vira "m-tree"). **Testado nesta consolidação (Seção 20.6): confirmação parcial, não geral.**

1.20.3 20.3 Correção real: medida uniforme vs. tesselação em múltiplas escalas (`BitmapQuadtree`)

O `stream::BitmapQuadtree` (Seção 18, dado real do bunny) foi implementado inicialmente com um scan célula a célula por nó — custo $O(\text{size}^2)$ por consulta de homogeneidade, quando o teste analítico da malha sintética (`SpatialPartitionExact`) é $O(1)$. Isso foi corrigido para uma **pirâmide booleana** (mipmap: OR-pool e AND-pool 2×2 por nível, pré-computada uma vez por fatia), que reduz cada consulta a 1 lookup direto — $O(1)$ de verdade.

A conexão com este relatório: o erro original era, em essência, usar a mesma "régua" (resolução de 1 célula) para decidir homogeneidade em qualquer escala de região — o oposto do princípio de tesselação em múltiplas resoluções já registrado na Seção 19 (um pedreiro não mede tijolo a tijolo contra a parede inteira; usa instrumentos de escala apropriada a cada distância). A pirâmide booleana é exatamente esse princípio aplicado: cada nível é uma tesselação mais grosseira da mesma fatia, e a

escala usada em cada consulta é a do nó que está sendo decidido, não sempre a mais fina.

Verificado por medição direta, não suposição: instrumentado com contadores reais de leitura/lookup (não estimativa teórica) — `tests/test_bitmap_pyramid.cpp` confirma 1024 leituras de bit para um nó de tamanho 32 e 16 para um nó de tamanho 4 na versão antiga (bate exato com 32^2 e 4^2), contra **1 lookup** em ambos os casos na versão nova — e paridade de resultado exata (mesma contagem de nós) nas 6 fatias do bunny testadas.

1.20.4 20.4 Segunda correção: a pirâmide ainda era força bruta disfarçada

Um ponto adicional foi levantado sobre a Seção 20.3: a pirâmide booleana é $O(1)$ por *consulta*, mas para *construir* a pirâmide ela ainda toca todas as $O(L^2)$ células da grade, inclusive as vazias — não usa a esparsidade conhecida do dado. Medido: para o bunny em L=32 (9,5% ocupado), a pirâmide toca 43.680 células contra apenas 3.112 voxels de dado real — 14x de desperdício. Isso é, na prática, "medir uniformemente contra uma grade completa" em vez de subtrair de uma referência conhecida (o princípio da Seção 13/14 deste relatório), ou de tesselar em múltiplas escalas respeitando a estrutura local dos dados (Seção 19) — mais próximo de uma calçada tesselada sem padrão único, cuja variação local não é bem servida por uma régua de escala fixa.

Correção: `stream::SpatialPartitionSparse` constrói a árvore partindo só da lista de pontos ocupados, nunca materializando a grade densa. Uma primeira tentativa de implementação tinha um bug real (medido, não suposto): filtrava a lista de pontos 4 vezes por nó, resultando em **mais** trabalho (60.440) que a própria pirâmide densa. Corrigido para particionamento em 1 única passada por nó: **17.444** pontos examinados — 2,5x melhor que a pirâmide.

Validado em duas escalas de esparsidade (L=32 e L=64), confirmando que a vantagem cresce com a esparsidade, não é um efeito isolado de um único caso de teste:

L	ocupação	trabalho esparso/pirâmide	economia
32	9,50%	0,3994	60,1%
64	4,71%	0,2350	76,5%

Achado colateral sobre espaço em disco (não trabalho de construção): o formato esparso ocupa mais bytes em disco que o bitmap denso nesta faixa de ocupação (denso custa $L^2/8$ bytes fixos; esparso custa ≈ 4 bytes/ponto). O ponto de equilíbrio teórico é $p = 1/32 = 3{,}125\%$ de ocupação — abaixo disso o esparso deveria ganhar em espaço também. Confirmado com os dois pontos de dado real disponíveis (calculados de forma independente da fórmula): L=32 prevê 3,09%, L=64 prevê 3,11% — ambos consistentes com a previsão teórica.

1.20.5 20.5 Resultado negativo: Morton code + busca binária não compensa aqui

Tentativa de reduzir ainda mais o custo da construção esparsa (cada ponto é examinado ~5,6-6,6x, uma vez por nível de profundidade da árvore): ordenar os pontos por

código Morton (Z-order) permite que cada divisão de quadrante vire busca binária $O(\log N)$ em vez de reparticionar a lista inteira — a propriedade de que pontos numa região alinhada a potência de 2 formam, em ordem Morton, um intervalo contíguo (e os 4 quadrantes formam 4 sub-blocos contíguos exatos) foi validada separadamente (0 falhas em 200 casos aleatórios) antes de medir o ganho.

Contando de forma justa (comparações elementares, incluindo o custo do `sort` inicial, que a versão sem Morton não paga): o resultado é **pior**, não melhor — 2,25x (L=32) e 2,15x (L=64) mais trabalho que o particionamento simples da Seção 20.4. Causa: o `sort` custa $O(N \log N)$ e é pago do zero a cada fatia, sem consultas repetidas que amortizem esse custo — a árvore de cada fatia é construída uma única vez, não consultada muitas vezes com a mesma ordenação.

Mantido no código como resultado negativo documentado (mesmo padrão do esquema híbrido da Seção 17) — não adotado em produção. A via mais promissora ainda não testada seria reaproveitar a ordenação Morton entre frames consecutivos, já que fatias vizinhas se repetem no ring buffer conforme a câmera se move gradualmente.

1.20.6 20.6 A hipótese do m-tree (Seção 20.2), testada: confirmação parcial e delimitada

A hipótese registrada na Seção 20.2 — subdividir em m partes por eixo em vez de sempre 2, alinhado ao k/L típico dos dados — foi implementada e medida (`py/verifica_mtree.py`), generalizando o teste de homogeneidade geométrico exato (Seção 9.1) de subdivisão binária para m-ária.

Primeira medição teve um viés real, corrigido antes de reportar: com $L = 256$ fixo para todo m testado, $m = 2$ venceu em todos os casos — mas $m \neq$ potência de 2 exige preencher a grade até a próxima potência de m (ex.: $m = 3$ precisa de grade 729 para $L = 256$, quase 3x maior), o que infla nós por preenchimento, não pela hipótese em si. Corrigido usando $L = m^{\text{depth}}$ exato para cada m (sem sobra).

Com o viés removido, o resultado mudou: para $k/L = 1/3$ especificamente (o pior caso do quadtree, Seção 9.1), $m = 3$ **venceu** — nós/L $= 4{,}486$ contra $6{,}535$ do quadtree. Faz sentido algebricamente: $1/3 = 3^{-1}$ é alinhado à base 3, do mesmo jeito que $1/4 = 2^{-2}$ e $1/2 = 2^{-1}$ são alinhados à base 2.

Mas isso não generaliza, testado com as frações "naturais" correspondentes para outros valores de m:

fração	nós/L (m=2)	nós/L (m natural)	m natural vence?
1/3 (m=3)	6,535	4,486	**sim**
1/9 (m=3)	4,332	4,486	não
1/5 (m=5)	4,707	6,242	não
1/6 (m=6)	5,223	7,338	não
1/7 (m=7)	4,520	8,146	não

Só o caso $m = 3, k/L = 1/3$ venceu de fato. Para os demais, o quadtree continua melhor — o fator de ramificação maior (m^2 filhos por nó não-homogêneo, em vez de 4) custa mais do que o alinhamento economiza, na maioria dos casos testados.

Conclusão: a hipótese estava certa em princípio — existe pelo menos um caso real onde m-tree bate o quadtree — mas era otimista em escopo: não é uma melhoria geral

aplicável trocando m para "casar" com qualquer k/L. Não portado para a engine C++ (resultado exploratório; caracterizar precisamente quando compensa exigiria mais investigação, não feita aqui).

1.20.7 20.7 Ponto de fuga como caso particular de tesselação: por que a engine mede em vez de assumir

A Seção 19.5 já havia estabelecido que uma escala geométrica repetida ($x_i = x_0 r^i$, $y_i = y_0 - di$) **não** produz um ponto de fuga projetivo clássico — os pontos só ficam colineares em $(\ln x, y)$, com inclinação medida $-5{,}7708$. Essa construção tem exatamente a mesma forma algébrica da escala entre níveis de uma tesselação hierárquica: o quadtree subdivide por potências de 2 (2^i entre níveis), o m-tree da Seção 20.6 subdivide por potências de m (m^i entre níveis) — ambos fatores de escala repetidos geometricamente, estruturalmente idênticos à Rota B.

A consequência prática: se a engine assumisse, ao projetar objetos entre níveis de LOD/tesselação, que a escala aparente segue um ponto de fuga clássico (linear em (x, y)), estaria assumindo o modelo errado por construção — o padrão real de uma escala geométrica repetida só é linear em $(\ln x, y)$. Por isso `render::VanishingPointAnalyzer` (Seção 18) **mede** qual dos dois padrões (linear ou log-linear) ocorre de fato nos dados correntes da janela ativa, em vez de assumir um dos dois — o mesmo princípio já usado pelo `Core::HardwareProfiler` (mede RAM real via `/proc/meminfo` em vez de assumir um valor fixo, Seção 18) e pela decisão de base m da subdivisão (Seção 20.6, decidida por medição, não suposição).

Validação end-to-end, não apenas unitária: integrado ao `ProjectivePipeline::Draw`, o detector foi testado contra o `world.stream` sintético real da engine. Achado no processo: a primeira integração sempre detectava um caso degenerado (variância zero nos dois modelos), porque a `Slice` expunha sempre `L` (constante) como "escala aparente", não `k` (que de fato varia com z nesse formato: `k = k_{max} - z`, Seção 5). Corrigido para expor `k`. Como esse decréscimo é linear por construção (1 unidade por fatia), a previsão matemática verificável era detectar o modelo **linear** com inclinação $-1{,}0000$ — e foi exatamente o resultado obtido em todos os frames testados, confirmando que a integração mede corretamente, não apenas não trava.

1.20.8 20.8 O scroll é um cilindro: homografia local, projeção cilíndrica global

O `Stream::WindowController` (ring buffer/scroll, Seção 3.2 da especificação original) já é, geometricamente, um anel: cada posição corresponde a um ângulo ao redor de um centro fixo — um cilindro. Isso tem uma consequência matemática precisa: quando uma câmera gira em torno do **mesmo centro óptico**, a relação entre duas capturas em ângulos diferentes é sempre uma **homografia** $H = K \cdot R \cdot K^{-1}$ (resultado clássico de visão computacional, usado em costura de panoramas). Isso foi implementado e validado antes de qualquer uso: composição $H(\theta_1) \cdot H(\theta_2) \approx H(\theta_1 + \theta_2)$ confirmada com erro $\sim 10^{-14}$, paridade exata entre Python e C++.

Limite real da homografia de plano único, medido, não suposto: $\det(H)$ explode conforme $\theta \to 90°$ (medido: $0{,}92$ em $10°$, $73{,}7$ em $91°$) — uma homografia de UM plano só não consegue representar um panorama de $360°$. Por isso a engine usa duas projeções para papéis diferentes:

- **Homografia** (`render::Homography`) — só para alinhar cada fatia contra sua vizinha imediata (ângulo pequeno, $2\pi/m$).
- **Projeção cilíndrica** (não-linear, `render::CylindricalPhotoCompiler`) — para posicionar cada fatia no panorama global, sem o problema de degeneração perto de $90°$.

O trade-off "modo foto pausa o jogo", implementado de verdade: compilar a foto (janela inteira do anel de uma vez) e desenhar o frame normal por frame são operações mutuamente exclusivas sobre o mesmo framebuffer. Isso foi testado por contagem exata, não apenas documentado: numa simulação de 30 frames com gatilho de foto a cada 10, exatamente 28 frames tomaram o caminho normal (`Draw`) e 2 tomaram o modo foto (`Compile`), $28 + 2 = 30$, sem nenhum frame fazendo os dois.

Validação visual com dado real do asset sintético: a primeira tentativa de gerar o panorama produziu uma imagem com todo o conteúdo espremido num canto — não por erro de matemática, mas porque a demonstração usou poucos frames simulados contra o anel de tamanho real ($m = 1024$, medido pelo `HardwareProfiler`, Seção 18), deixando a maior parte do anel com valores padrão (vazios). Corrigido preenchendo o anel por completo; o resultado mostra 4 repetições idênticas do mesmo padrão visual — consistente com $1024/256 = 4$, o ciclo do parâmetro $k = k_{max} - z$ do asset sintético (Seção 5) se repetindo exatamente 4 vezes ao preencher o anel. Isso confirma que a projeção reflete o dado real subjacente corretamente, não é um artefato visual.

1.21 21. Números primos como interseção de triângulos: completando a Seção 19

O documento "Números Primos como Interseção de Triângulos Semelhantes" retoma exatamente a construção da Seção 19.1 ($N = T_{k+1} - T_{k-1}$, o caso $h = 2$ da decomposição trapezoidal) e completa a peça que faltava lá: **essa decomposição é única precisamente quando N é primo**. Esta seção incorpora esse resultado e as extensões geométricas que o acompanham (números hexagonais via triangulação real, múltiplas tesselações sobre a mesma malha, e a dualidade cubo/octaedro), todas verificadas nesta consolidação por reconstrução computacional independente — não apenas aceitas por leitura.

1.21.1 21.1 Critério de unicidade e primalidade (completa a Seção 19.1)

Para N ímpar, $N > 1$, uma decomposição trapezoidal de altura $h = b - a$ satisfaz $T_b - T_a = N \iff (b - a)(b + a + 1) = 2N$. A altura $h = 1$ é sempre trivial ($N = T_N - T_{N-1}$); o objeto de interesse são as decomposições com $h \geq 2$.

Teorema (unicidade para primos): N é primo $\iff$ existe exatamente uma decomposição não trivial, e nesse caso $h = 2$ sempre, com $N = T_{k+1} - T_{k-1}$, $k = (N - 1)/2$ — exatamente a fórmula da Seção 19.1, agora com o critério que faltava. A demonstração usa que $2N = h \cdot m$ (com $m = b + a + 1$) exige h, m de paridades opostas; o número de decomposições não triviais é igual ao número de divisores ímpares de N maiores que 1, que é exatamente 1 se e somente se N é primo.

Verificado nesta consolidação por busca exaustiva (não apenas conferência da fórmula): confirmado para todo N ímpar de 3 a 200, incluindo a tabela completa do

documento ($N = 5, 7, 9, 11, 15, 19, 21$, com as decomposições (a, b) exatas batendo em todos os casos) e a relação "nº decomposições = nº divisores ímpares > 1" testada até $N = 200$. Compostos admitem múltiplas faixas (uma por divisor ímpar não trivial: $N = 9$ tem 2, $N = 15$ e $N = 21$ têm 3), primos admitem exatamente uma, com $h = 2$ sempre.

1.21.2 21.2 Pirâmide simétrica e números hexagonais (complementar, não ligada a primalidade)

A pirâmide simétrica $(1, 2, \ldots, p-1, p, p-1, \ldots, 1)$ soma p^2 — outra via geométrica para a identidade clássica $\sum_{i=1}^{p}(2i-1) = p^2$, já conectada à Seção 19.2 deste relatório (gnômon). Números hexagonais centrados, $H(k) = 3k^2 + 3k + 1$, contam os pontos de um hexágono de raio k na malha triangular verdadeira (6 vizinhos equidistantes por ponto), e se reconstroem a partir do trapézio $T_{2k} - T_k$ por espelhamento: $H(k) = 2(T_{2k} - T_k) + (2k + 1)$ — a mesma matemática de diferença de triangulares da Seção 19.1/21.1, aplicada a uma figura diferente (hexágono, não faixa reta).

Verificado nesta consolidação: identidade da pirâmide (soma $= p^2$) confirmada para $p = 1..29$; $H(k)$ confirmado tanto pela construção por anéis concêntricos quanto pela fórmula fechada, para $k = 0..29$; espelhamento do trapézio confirmado idêntico a $H(k)$ nos mesmos casos.

1.21.3 21.3 Vãos e partições: o mesmo conjunto de pontos, múltiplas tesselações

Além de contar vértices, a malha triangular define **vãos** (faces): o hexágono de raio k (com $H(k)$ vértices) se particiona em exatamente $6k^2$ triângulos elementares. **Verificado nesta consolidação por triangulação de Delaunay real** (não recontagem da fórmula): construída a malha hexagonal explicitamente para $k = 1, 2, 3, 4$, triangulada via `scipy.spatial.Delaunay`, filtrados os triângulos de área unitária equilátera — resultado exato $6, 24, 54, 96$ em todos os casos, batendo $6k^2$.

Esse mesmo conjunto de pontos e vãos admite mais de um reagrupamento válido: pares de triângulos adjacentes formam losangos (máximo teórico $\lfloor 6k^2/2 \rfloor = 3k^2$, mas emparelhamento guloso simples fica abaixo disso, com triângulos remanescentes sem par — o documento reporta 25 de 27 possíveis para $k = 3$); os 6 triângulos ao redor de cada vértice interno formam um hexágono regular, e o número de hexágonos completos que cabem é $H(k-1)$ (verificado: $H(2) = 19$ bate exato). Isso é a mesma família de resultado da Seção 19.3 deste relatório (tesselação por losangos, verificada por rasterização independente) — aqui generalizada: a malha triangular subjacente é única, mas as construções que se apoiam nela (losango, hexágono, triângulo elementar) não são, e nem todo reagrupamento atinge o máximo teórico, exatamente como a Seção 19.3 já havia registrado sobre tesselação em geral.

1.21.4 21.4 Do hexágono aos sólidos: cubo e octaedro (nova conexão, verificada)

O hexágono regular, dividido pelos 3 eixos internos a partir do centro, produz 3 losangos congruentes (ângulos $60°/120°$, somando $3 \times 120° = 360°$) — reconhecido como a ilusão óptica clássica do "cubo de Necker isométrico". Cubo e octaedro são sólidos

duais; projetados ao longo do eixo de simetria de ordem 3, ambos produzem o mesmo contorno hexagonal, mas particionado diferente: 3 losangos (cubo) ou 6 triângulos (octaedro).

Verificado nesta consolidação, não só aceito por leitura: construí um octaedro regular explícito (vértices $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$, $(0, 0, \pm 1)$) e projetei os 6 vértices no plano perpendicular ao eixo $(1, 1, 1)/\sqrt{3}$ — as 6 distâncias ao centro projetado deram exatamente iguais ($0{,}8165$ em todas), e as 6 diferenças angulares entre vértices consecutivos deram exatamente $60°$ em todas — confirmando por geometria 3D direta (não por aceitar a afirmação) que a sombra de um octaedro nesse eixo é mesmo um hexágono regular perfeito, não uma aproximação.

1.21.5 21.5 Pontos de fuga (Seção 7 do documento): verificado de forma independente

O documento trata pontos de fuga como **linha de investigação separada**, sem conexão formal com a aritmética de primos — postura correta, mantida aqui. A conexão real que existe é com o `render::VanishingPointAnalyzer` já registrado na Seção 20.7 deste relatório: ambos tratam de medir (não assumir) o comportamento de convergência/escala em projeção.

Verificado nesta consolidação com câmera sintética própria (não os números exatos do documento, que dependem da orientação específica de câmera deles — testei os *fenômenos*, de forma independente):

Fenômeno afirmado	Meu teste independente	Confirma?
Direções genuinamente paralelas convergem com erro $0°$	erro $= 0{,}0000°$	Sim
Desvio angular introduzido causa divergência real	$8°$ de desvio → $8°$ de divergência medida	Sim
Pitágoras ingênuo falha em coordenadas de imagem mesmo com ângulo 3D real $= 90°$	ângulo ingênuo $= 96{,}86°$ (não $90°$), discrepância $6{,}84\%$ (documento: $92{,}66°$/$4{,}12\%$, câmera diferente)	Sim, mesmo fenômeno, números diferentes por setup diferente
Ângulo correto só se recupera normalizando por K^{-1}	recuperado $= 90{,}0000°$ exato	Sim

Os números absolutos diferem do documento (orientações de câmera distintas, esperado), mas os quatro fenômenos qualitativos se confirmam de forma totalmente independente — reforça que a ressalva do documento (geometria de imagem bruta não é euclidiana da forma ingênua) é um fato real de visão computacional, não peculiaridade do experimento deles.
